

# Flow control makes shared inference predictable.

Flow control is the Endpoint Picker's policy layer for multi-tenant inference. While the pool has capacity, requests move through without being held. When the pool is under pressure, flow control activates. New work waits in policy-aware queues in front of the model servers, so priority, fairness, and tenant rules decide what runs next.

This page explains the mechanism end to end. What breaks without it, how a request travels through it, what it guarantees, what it costs, and how to operate it.

## Why LLM traffic breaks rate limiting

Traditional gateways limit requests per second and assume every request costs about the same. LLM requests break that assumption.

Cost is driven by input length and by the autoregressive decode loop. One request can consume orders of magnitude more compute and memory than another at the same request rate.

### Same request rate, very different cost

256-token prompt, 64-token reply

occupies a batch slot briefly

2048-token prompt, 512-token reply

occupies compute and KV cache for the whole decode loop

Measured on one GPU at 160 concurrent requests, the short shape cleared 165 requests per second and the long shape cleared 27. A request-per-second limit cannot see this difference. Physical capacity can.

### Noisy neighbor

One tenant with heavy prompts monopolizes KV cache and queue space, starving the rest.

### Scheduling regret

Once a request enters a server's local queue it cannot move, even when a better endpoint frees up.

### Priority inversion

Model servers batch by arrival, not business priority. Real-time work gets stuck behind offline batches.

### Resource asymmetry

One huge-context request can outweigh hundreds of small ones. Request counts miss it.

Flow control answers by governing physical capacity, KV cache and queue saturation, instead of request counts, and by holding the queue where policy can still act on it.

## What flow control is, and what it is not

### WHAT IT IS

A centralized, policy-aware queue in the Endpoint Picker, in front of the model servers. It enforces priority across traffic

### WHAT IT IS NOT

More capacity. Flow control decides when and in what order requests dispatch, and it cannot make the pool faster.

## WHAT IT IS

classes, fairness across tenants, and ordering within a tenant.

It is work-conserving. If the pool has capacity, it dispatches.

It routes late. A dispatched request lands on the endpoint that just freed up or holds its prefix cache.

## WHAT IT IS NOT

When demand exceeds capacity, the wait still exists. Flow control moves it into the EPP queue and decides where and for whom time to first token is accrued, protecting generation speed for everything already running.

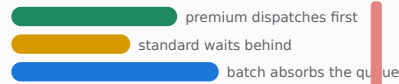
### The two states of the system

#### POOL HAS CAPACITY · gate open



Requests move straight through. Flow control is invisible.

#### POOL SATURATED · gate closed



New work queues by band. The policy decides who goes next.

That trade is the honest core of the system. Enabling flow control is an explicit choice to protect time per output token, the reading pace of every stream already in flight, at the price of queue time for new arrivals when the pool is full.

## The journey of a request

Five steps between the client and the GPU. The first two classify, the middle two decide, the last one routes.

### 1 · TAG

#### The trusted gateway labels traffic

x-llm-d-inference-objective → priority  
x-llm-d-inference-fairness-id → tenant

### 2 · CLASSIFY

#### FlowKey = (tenant, priority)

Every FlowKey gets its own queue.  
Missing objective defaults to priority 0.

Trust boundary: clients never self-assert these headers. The gateway injects them from identity.

### 3 · QUEUE BY POLICY

#### Priority bands, then fairness, then order

#### Band 100 · premium · dispatches first

tenant-a queue tenant-b queue

fairness cycles tenants · ordering sorts each queue

#### Band 0 · standard · waits behind premium

tenant-c queue

#### Band -10 · sheddable · bounded queue

batch queue

Per-band request and byte limits protect the gateway's memory. Past a band's limit, arrivals get HTTP 429.  
A queued request that outlives its TTL gets HTTP 503.

### 4 · THE GATE

#### Saturation detector

Pool has room → dispatch.  
Pool saturated → hold everything, re-check each cycle.  
Reads queue depth and KV cache, or in-flight request count.

### 5 · LATE BINDING

#### The scheduler picks late

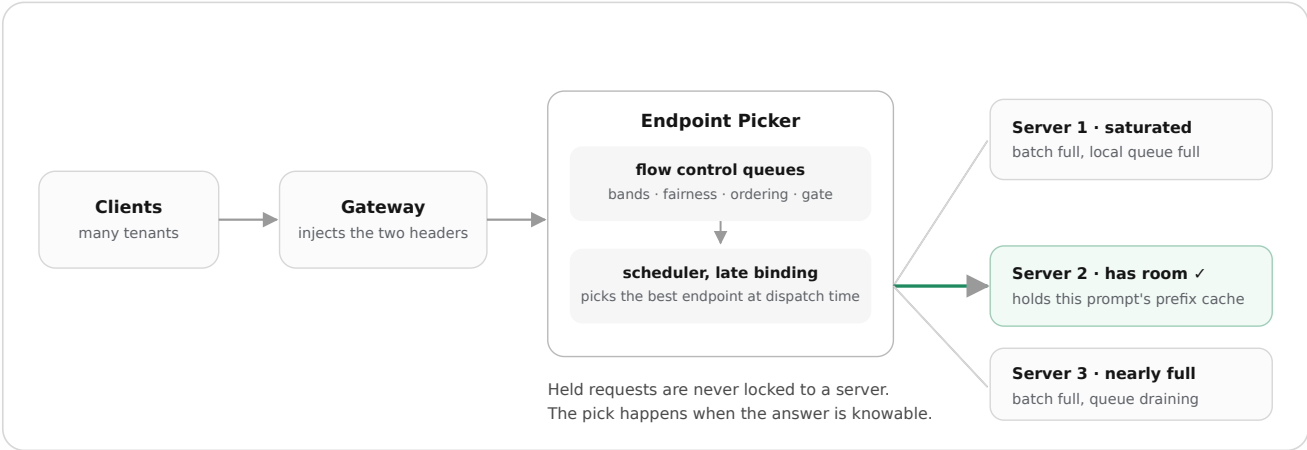
The endpoint is chosen at dispatch, where capacity or prefix cache is.

### vLLM model servers

Local queues stay a small healthy buffer, deep enough that continuous batching never starves, shallow enough that the real queue stays where policy lives.

## The architecture, one level up

The journey above happens inside one Endpoint Picker serving a pool of model servers. Late binding is why the queue lives centrally. The endpoint is chosen at dispatch, when the picker knows which server just freed capacity or already holds the request's prefix cache.



## The three-tier decision

Each dispatch cycle answers three questions in strict order. One is fixed, two are pluggable.

| TIER                | QUESTION                        | WHO DECIDES                               | OPTIONS                                                                               |
|---------------------|---------------------------------|-------------------------------------------|---------------------------------------------------------------------------------------|
| 1 · <b>Priority</b> | Which band goes first?          | Fixed. Highest band with work always wins | Bands come from <code>InferenceObjective</code> , negative values mark sheddable work |
| 2 · <b>Fairness</b> | Which tenant inside the band?   | Fairness policy plugin                    | Round-robin across tenants, or global strict ordering                                 |
| 3 · <b>Ordering</b> | Which request from that tenant? | Ordering policy plugin                    | First-come first-served, earliest deadline first, or SLO-deadline                     |

Priority is absolute by design. When the pool is saturated, every premium request dispatches before any standard request, and fairness only referees between equals.

If two workloads should compete, put them in the same band. If one should never wait for the other, put them in different bands.

## Where the waiting goes

When demand exceeds capacity, the wait has to land somewhere. There are only two places, and only one of them obeys policy.

Two waits, one budget

THE EPP QUEUE · policy decides

Time here is assigned. Priority picks the band, fairness picks the tenant, and the tenant that caused the pressure carries the most of it. Measurable per tenant and per band.

llm\_d\_epp\_flow\_control\_request\_queue\_duration\_seconds

THE RUNNING BATCH · physics decides

Once dispatched, a request computes inside the shared batch. A full batch produces first tokens more slowly for every request in it, whoever sent them. No scheduler reaches inside.

vllm:num\_requests\_running, vllm:num\_requests\_waiting

Tuning the saturation gate moves the wait between these two places. That is the dial. Let the batch fill for maximum utilization, or hold work upstream for maximum isolation, and choose per pool.

What the policy guarantees

Each tier leaves a fingerprint you can measure on your own pool.

PRIORITY

A latency-sensitive class stays fast through a surge it did not cause. The lower class absorbs the queue.

FAIRNESS

Tenants at equal load within a band are treated identically, without anyone tuning per-tenant rate limits.

COST ASSIGNMENT

The tenant that creates the pressure carries the most queue time. The wait is assigned, not ambient.

SHEDDABLE ISOLATION

Batch traffic can dominate arrivals and still absorb the waiting instead of causing it, inside a bounded queue.

Absolute latencies always track your hardware, model, and tuning. The pattern is the guarantee.

Targets hold while capacity lasts. Past capacity the policy, not arrival order, decides who waits.

When requests are refused

Flow control fails loudly and specifically. Every refusal carries a reason on the wire in the `x-llm-d-request-dropped-reason` header.

| WHAT HAPPENED                                          | STATUS | REASON ON THE WIRE                      |
|--------------------------------------------------------|--------|-----------------------------------------|
| A band's queue limit was hit, controlled load shedding | 429    | <code>rejected-saturated</code>         |
| A request waited past its TTL                          | 503    | <code>rejected-ttl-expired</code>       |
| The client disconnected while queued                   | 503    | <code>rejected-context-cancelled</code> |
| Graceful shutdown drained the queue, retryable         | 503    | <code>rejected-shutting-down</code>     |

Queues are in-memory. If the EPP itself dies, the default llm-d configuration fails open, Envoy routes directly to the servers, and requests keep flowing without protection until the EPP returns.

Operating it

The knobs that matter

**Band priorities** in `InferenceObjective` map business tiers to dispatch order. **Per-band limits** bound how much

What to watch

`pool_saturation` tells you when the gate is active.

low-priority work can queue.

**Request TTL** caps how stale queued work can get.

**The saturation detector's threshold** sets the healthy buffer, deep enough that batching never starves, shallow enough that the queue stays where policy lives. It is a trigger, not a queue cap.

`queue_size` and `queue_duration` per tenant and band tell you who is waiting and for how long. The same depth is a true-demand signal an autoscaler can act on.

On the server side, `num_requests_running` against `num_requests_waiting` tells you where the pressure actually sits.

One sweep per hardware and model pair finds the knee, the point where queueing begins. Size latency-sensitive pools below it, and let flow control govern everything that arrives past it.

That is the whole operating model. Know your envelope, place your operating point, and let the policy own the overflow.